

COMPTE RENDU R102

Dans Wireshark, quand tu développes la partie **Internet Control Message Protocol**, tu vois :

- **Type = 8** → Echo Request (ping envoyé)
- **Type = 0** → Echo Reply (réponse au ping)
- **Type = 3** → Destination Unreachable
- **Type = 11** → Time Exceeded

Type	Nom du message	Rôle / Signification principale
0	Echo Reply	Réponse à un ping (retour du paquet)
3	Destination Unreachable	La destination n'est pas atteignable (ex : réseau ou port inaccessible)
5	Redirect	Indique qu'il faut utiliser une autre passerelle pour atteindre la destination
8	Echo Request	Requête ping envoyée par une machine
9	Router Advertisement	Un routeur annonce sa présence
10	Router Solicitation	Une machine demande à découvrir un routeur
11	Time Exceeded	TTL expiré (le paquet a fait trop de sauts)
12	Parameter Problem	Erreur dans l'en-tête IP (champ invalide)

Type	Nom du message	Rôle / Signification
0	Echo Reply	Réponse à un ping
3	Destination Unreachable	Destination inaccessible
5	Redirect	Redirection vers une autre passerelle
8	Echo Request	Requête ping
9	Router Advertisement	Routeur annonce sa présence
10	Router Solicitation	Machine demande un routeur
11	Time Exceeded	TTL expiré
12	Parameter Problem	Erreur dans l'en-tête IP

Opération	Nom	Rôle
1	Request (who-has)	Une machine demande « qui a cette IP ? »
2	Reply (is-at)	La machine répond « cette IP est à cette MAC »

👉 Encapsulation ARP :

Code ^

Copier

Ethernet II → ARP

♦ Ports TCP/UDP des protocoles applicatifs

Protocole	Port(s)	Transport	Sécurisé ?
DNS	53	UDP/TCP	Non (DNSSEC existe)
DHCP	67 (serveur), 68 (client)	UDP	Non
SSH	22	TCP	Oui
FTP	21 (contrôle), 20 (données)	TCP	Non (SFTP/FTPS existent)
HTTP	80	TCP	Non
HTTPS	443	TCP	Oui

Protocole	Encapsulation
ICMP	Ethernet → IP → ICMP
ARP	Ethernet → ARP
DNS	Ethernet → IP → UDP → DNS
DHCP	Ethernet → IP → UDP → DHCP
SSH	Ethernet → IP → TCP → SSH
FTP	Ethernet → IP → TCP → FTP
HTTP	Ethernet → IP → TCP → HTTP
HTTPS	Ethernet → IP → TCP → TLS → HTTP

◆ Commandes utilisées (récapitulatif complet)

TP1 – ICMP

- `ping -c1 192.31.25.1` → envoi d'un ping unique.
- `ping -c3 google.com` → ping avec 3 paquets.
- `ping -i 0.2 192.31.25.1` → ping avec intervalle de 0,2s.
- `ping -t 5 192.31.25.1` → ping avec TTL fixé à 5.
- `ping -s 1000 192.31.25.1` → ping avec taille de paquet personnalisée.

TP2 – ARP

- `ip neigh show` → afficher le cache ARP.
- `ip neigh flush all` ou `sudo ip n f dev enp0s3` → vider le cache ARP.
- `ping 192.31.25.1` → déclenche une requête ARP si MAC inconnue.

TP3 – Protocoles applicatifs

- `nslookup www.exemple.com` → requête DNS.
- `ftp iut-rt` → connexion FTP.
- `ssh guest00@iut-rt` → connexion SSH.
- `telnet iut-rt 23` → connexion Telnet (si activé).
- `curl http://iut-rt.univ-artois.fr` → requête HTTP.
- `dhclient -v` → requête DHCP (client).

TP4 – Scapy

- `scapy` → lancer Scapy.
- `IP(dst="192.31.25.1")/ICMP()` → créer un paquet ICMP.
- `send(IP(dst="192.31.25.1")/ICMP())` → envoyer un paquet ICMP.
- `wrpcap("fichier.pcap", paquet)` → écrire un paquet dans un fichier.
- `rdpcap("fichier.pcap")` → lire un fichier pcap.

Checklist d'analyse

1. Encapsulation (ETH → IP → ...)
2. Ethernet (MAC src/dst, type)
3. IP (IP src/dst, TTL, protocole)
4. Transport (ports, flags, type ICMP/ARP)
5. Application (DNS, HTTP, FTP...)
6. Conclusion (rôle de la trame)

◆ Étapes pour réaliser l'exercice de Port Knocking

1. Vérifier l'état du port SSH

- Sur ta machine cliente (Ubuntu), essaye de te connecter en SSH :

Bash ^

 Copier

```
ssh guest00@iut-rt
```

- Résultat attendu : **connexion refusée** car le port 22 est fermé par défaut.

2. Envoyer la séquence de knocks

 In

- Utilise la commande `knock` (si elle est installée) ou bien `nmap` pour envoyer des paquets vers les ports définis dans le TP.
- Exemple avec `knock` :

Bash ^

 Copier

```
knock iut-rt 7000 8000 9000
```

- Ici, la séquence est **7000 → 8000 → 9000** (elle peut varier selon le TP).
- Chaque "knock" est une tentative de connexion sur un port fermé, enregistrée par le démon `knockd` côté serveur.

3. Vérifier l'ouverture du port

- Après la séquence correcte, le pare-feu ouvre temporairement le port SSH (22).
- Tu peux vérifier avec un scan :

Bash ^

 Copier

```
nmap -p 22 iut-rt
```

- Résultat attendu : le port 22 apparaît comme **open**.

4. Connexion SSH

- Maintenant, tu peux te connecter en SSH :

Bash ^

 Copier

```
ssh guest00@iut-rt
```

- Résultat attendu : la connexion fonctionne, car le port est ouvert.

5. Fermeture automatique

- Après un délai (souvent quelques secondes ou minutes), le port se referme automatiquement.
- Tu peux vérifier à nouveau avec :

Bash ^

 Copier

```
nmap -p 22 iut-rt
```

- Résultat attendu : le port 22 est redevenu **closed**.

◆ Ce que tu dois analyser dans Wireshark

- Les trames envoyées vers les ports 7000, 8000, 9000 (séquence knocking).
- La trame de connexion SSH une fois le port ouvert.
- Tu peux montrer l'encapsulation :
 - Knock → ETH → IP → TCP (vers port 7000/8000/9000)
 - SSH → ETH → IP → TCP (vers port 22)

EXERCICE TP 3 PORT KNOCKING

Objectif de l'exercice

Tu dois **trouver la séquence de ports** utilisée pour ouvrir le port SSH (22) sur la machine distante `172.17.123.177`, en analysant le fichier `port-knocking.pcap` fourni dans Moodle.

Ce qu'il faut comprendre

◆ Qu'est-ce que le port knocking ?

- C'est une **technique de sécurité** : le port SSH est **fermé par défaut**.
- Pour l'ouvrir, il faut "toquer" à une **séquence de ports fermés** (ex. 1234 → 2345 → 3456).
- Chaque "knock" est une **tentative de connexion TCP (SYN)** vers un port fermé.
- Si la séquence est correcte, le serveur ouvre temporairement le port 22 (SSH).

Étapes pour faire l'exercice

1. Ouvrir le fichier `port-knocking.pcap` dans Wireshark

- Fichier fourni dans Moodle → TP3 → `port-knocking.pcap`
- Ouvre-le dans Wireshark.

Inv

2. Filtrer les frames TCP SYN

Utilise ce filtre pour ne voir que les knocks :

Bash ^

 Copier

```
tcp.flags.syn == 1 and tcp.flags.ack == 0
```

👉 Ce filtre affiche uniquement les **tentatives de connexion TCP** (SYN sans ACK).

3. Identifier les ports frappés

- Regarde les **ports de destination** dans chaque frame TCP SYN.
- Note-les dans l'ordre d'apparition.
- Exemple :

Code ^

 Copier

```
Trame 1 → Port 1234  
Trame 2 → Port 2345  
Trame 3 → Port 3456
```

4. Trouver la séquence complète

- Tu dois **reconstituer la séquence exacte** utilisée pour ouvrir le port SSH.
- Elle est souvent composée de **3 knocks**.
- Une fois la séquence terminée, tu verras une frame vers le port 22 (SSH) → c'est la preuve que la séquence a fonctionné.

Objectif

Confirmer que le port 22 (SSH) s'est ouvert **après la séquence**.

Étapes dans Wireshark

1. Filtre pour voir les connexions vers le port 22 :

Bash ^

 Copier

```
tcp.dstport == 22
```

Étapes dans Wireshark

1. Ouvre le fichier `port-knocking.pcap` dans Wireshark.
2. Applique ce filtre pour ne voir que les knocks :

Bash ^

 Copier

```
tcp.flags.syn == 1 and tcp.flags.ack == 0
```

👉 Ce filtre affiche uniquement les **paquets SYN purs**, donc les knocks.

3. Regarde chaque frame filtrée :

- Note l'**IP source** (normalement celle du client).
- Note l'**IP destination** (celle du serveur : `172.17.123.177`).
- Note le **port de destination** → c'est le port frappé.

4. Note les ports dans l'ordre d'apparition :

- Trame 1 → port 1234
- Trame 2 → port 2345
- Trame 3 → port 3456

👉 Tu obtiens la **séquence complète** : `1234 → 2345 → 3456`

Objectif

Confirmer que le port 22 (SSH) s'est ouvert **après la séquence**.

Étapes dans Wireshark

1. Filtre pour voir les connexions vers le port 22 :

Bash ^

 Copier

```
tcp.dstport == 22
```

2. Regarde si tu vois une frame avec :

- **SYN** → demande de connexion
 - **SYN/ACK** → réponse du serveur
 - **ACK** → confirmation du client
- 👉 C'est le **3-way handshake TCP** → preuve que le port SSH est ouvert.

3. Vérifie que cette frame SSH vient juste après les knocks :

- Si oui → la séquence a fonctionné 
- Si non → soit la séquence est incorrecte, soit le port n'a pas été ouvert

Objectif

Confirmer que le port 22 (SSH) s'est ouvert **après la séquence**.

Étapes dans Wireshark

1. Filtre pour voir les connexions vers le port 22 :

Bash ^

 Copier

```
tcp.dstport == 22
```

2. Regarde si tu vois une frame avec :

- **SYN** → demande de connexion
 - **SYN/ACK** → réponse du serveur
 - **ACK** → confirmation du client
- 👉 C'est le **3-way handshake TCP** → preuve que le port SSH est ouvert.

3. Vérifie que cette frame SSH vient juste après les knocks :

- Si oui → la séquence a fonctionné 
- Si non → soit la séquence est incorrecte, soit le port n'a pas été ouvert

4. Note les ports dans l'ordre d'apparition :

- Trame 1 → port 1234
- Trame 2 → port 2345
- Trame 3 → port 3456

👉 Tu obtiens la **séquence complète** : 1234 → 2345 → 3456

2. Regarde si tu vois une trame avec :

- `SYN` → demande de connexion
 - `SYN/ACK` → réponse du serveur
 - `ACK` → confirmation du client
- 👉 C'est le **3-way handshake TCP** → preuve que le port SSH est ouvert.

3. Vérifie que cette trame SSH vient juste après les knocks :

- Si oui → la séquence a fonctionné ✅
- Si non → soit la séquence est incorrecte, soit le port n'a pas été ouvert

4. Note les ports dans l'ordre d'apparition :

- Trame 1 → port 1234
- Trame 2 → port 2345
- Trame 3 → port 3456

👉 Tu obtiens la **séquence complète** : 1234 → 2345 → 3456

5. Vérifier l'ouverture du port SSH

- Cherche une trame TCP vers le port 22 avec ce filtre :

Bash ^

📋 Copier

```
tcp.dstport == 22
```

- Si tu vois un **SYN → SYN/ACK → ACK**, c'est que le port SSH a été ouvert avec succès.

POUR RECONSTITUER LE PORT

🔧 Étapes dans Wireshark

1. Ouvre le fichier `port-knocking.pcap` dans Wireshark.
2. Applique ce filtre pour ne voir que les knocks :

Bash ^

📋 Copier

```
tcp.flags.syn == 1 and tcp.flags.ack == 0
```

👉 Ce filtre affiche uniquement les **paquets SYN purs**, donc les knocks.

3. Regarde chaque frame filtrée :

- Note l'**IP source** (normalement celle du client).
- Note l'**IP destination** (celle du serveur : `172.17.123.177`).
- Note le **port de destination** → c'est le port frappé.

4. Note les ports dans l'ordre d'apparition :

- Trame 1 → port 1234
- Trame 2 → port 2345
- Trame 3 → port 3456

👉 Tu obtiens la **séquence complète** : `1234 → 2345 → 3456`

🎯 Objectif

Confirmer que le port 22 (SSH) s'est ouvert **après la séquence**.

🔧 Étapes dans Wireshark

1. Filtre pour voir les connexions vers le port 22 :

Bash ^

📋 Copier

```
tcp.dstport == 22
```

2. Regarde si tu vois une frame avec :

- `SYN` → demande de connexion
 - `SYN/ACK` → réponse du serveur
 - `ACK` → confirmation du client
- 👉 C'est le **3-way handshake TCP** → preuve que le port SSH est ouvert.

3. Vérifie que cette frame SSH vient juste après les knocks :

- Si oui → la séquence a fonctionné ✅
- Si non → soit la séquence est incorrecte, soit le port n'a pas été ouvert

🧠 Astuce pour ne pas te perdre

- Tu peux **trier les frames par heure** (colonne "Time") pour bien suivre l'ordre.
- Tu peux aussi **ajouter une colonne "Port destination"** dans Wireshark pour les repérer plus vite.

